Computer Graphics

Shadows & Texture Mapping

Dr. Hardik Jain
Assistant Professor
Department of Computer Science and Engineering
IIT Jodhpur

Reference: CG Slides of Dr. Avinash Sharma

1

Shadow

- Shadows are the results of light not reaching some part of the scene.
- Surfaces that are not visible from the light sources are in shadow.
- VSD algorithms can be used to determine this:
  - VSD algorithms determine which surfaces can be seen from the viewpoint
  - Shadow algorithm determines which surface can be seen from the light source

2

Shadow Illumination Equation

- When a point on the surface cannot be seen from the a light source, then the illumination calculation must be adjusted to take it into account.
- Shadow illumination equation:

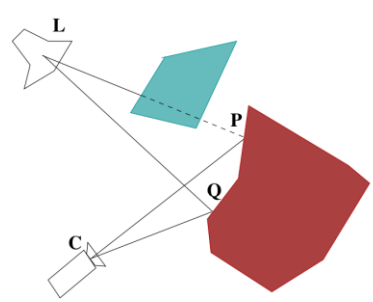
$$I_{s\lambda} = O_{\lambda a} k_a I_{\lambda a} + \sum_{l=1}^{n} S_l f_{att_l} I_{\lambda l} [k_d O_{\lambda d} (\mathbf{N} \cdot \mathbf{L}) + k_s O_{\lambda s} (\mathbf{V} \cdot \mathbf{R})^{n_s}]$$

$$S_l = \begin{cases} 0 & \text{if light } l \text{ is blocked at this point} \\ 1 & \text{if light } l \text{ is not blocked at this point} \end{cases}$$

3

2 Pass Z-Buffer Algorithm

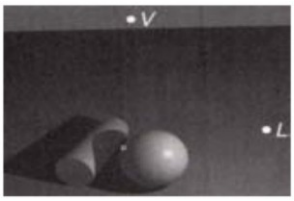
- Draw the scene with light source as the camera using Z-buffering.
- Create a **shadow map** by reading the depth values from the Z-buffer into a z_L buffer.
- Draw scene from camera's viewpoint with Z-buffering.
- Transform each visible point (x_o, y_o, z_o) to (x'_o, y'_o, z'_o) in the light source's coordinates.
- If $z'_o > z_L(x'_o, y'_o)$, there is another point that is closer to the light source. Do not light the pixel.



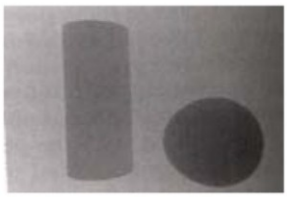
- Point P will be shadowed and Q will not be.

4

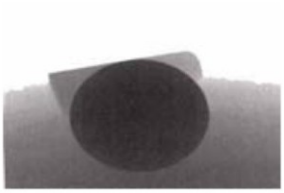
2 Pass Z-Buffer Algorithm



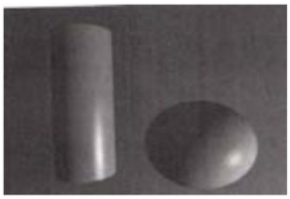
overview



Camera's Z Buffer



Light's Z Buffer



Observer's Image

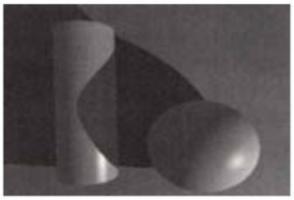


Image with Shadow

Computer Graphics, Principles & Practice in C by Foley & Van Dam

5

Spatially varying Surface Properties

- Attributes of surfaces that vary from place to place but don't really change the *shape* of the surface in a meaningful way.
- When trying to replicate the look of the real world, one quickly realizes that hardly any surfaces are featureless.
- Wood grows with grain; skin grows with wrinkles; cloth shows its woven structure; paint shows the marks of the brush or roller that laid it down.
- Even smooth plastic is made with bumps molded into it, and smooth metal shows the marks of the machining process that made it.

6

Texture Mapping

Mapping an image (texture map) to the surface, where the image stores the detail that you want to go on a surface

Texture

- Textures can be one, two, three, or four dimensional.
- For example, a one dimensional texture might be used to create a pattern for colouring a curve.
- A three dimensional texture might describe a solid block of material from which an object could sculpt.
- Because the use of surfaces is so important in computer graphics, mapping two-dimensional textures to surfaces is by far the most common use of texture mapping
- However, the processes of mapping is the same regardless of the dimensionality of the texture.

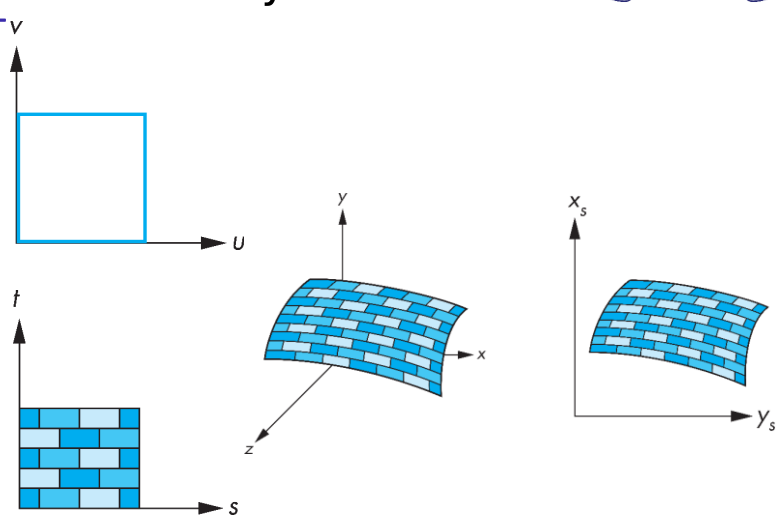
Texture Map

- A continuous rectangular two-dimensional texture map/pattern is denoted by array $T(s,t)$, elements of which are referred to as **texels**.
- The independent variables s and t are known as **texture coordinates**, which can be scaled to vary over the interval $[0,1]$
- A texture map associates a texel with each point on a geometric object that is itself mapped to screen coordinates for display.
- If the object is represented in homogeneous or *(x, y, z, w)* coordinates, then there are functions such that

$$x = x(s, t)$$
$$y = y(s, t)$$
$$z = z(s, t)$$
$$w = w(s, t)$$

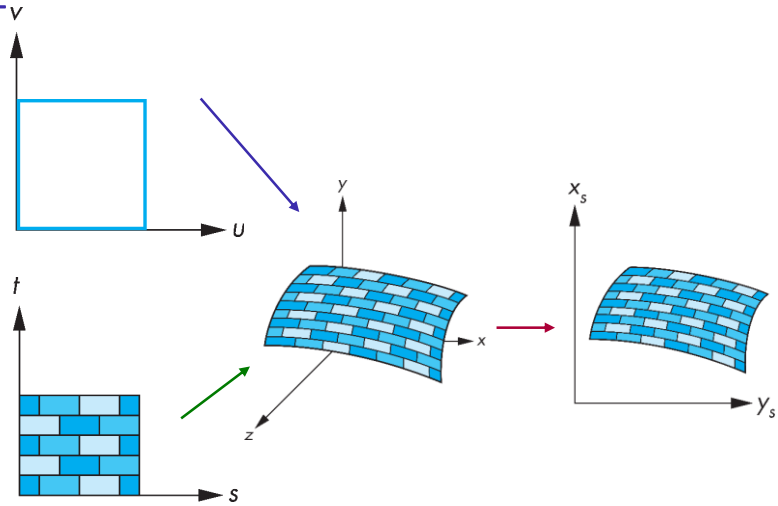
Texture Mapping: Coordinate Systems

- Parametric coordinates (u, v)
  - May be used to model curves and surfaces
- Texture coordinates (s, t)
  - Used to identify points in the image to be mapped
- Object or World Coordinates (x, y, z)
  - Conceptually, where the mapping takes place
- Window/Screen Coordinates (x_s, y_s)
  - Where the final image is really produced



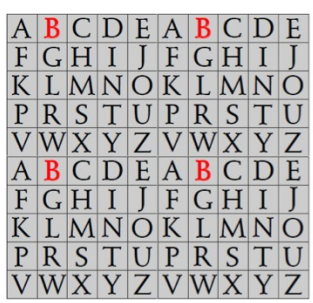
Texture Mapping: Procedure

1. From texture coordinates to object coordinates
2. From parametric coordinates to object coordinates
3. Mapping takes us from object coordinates to screen coordinates



Texture Mapping

- A small area of the texture pattern maps to the area of the geometric surface, corresponding to a pixel in the final image.
- Assuming that the values of *T* are RGB color values, either of the two colouring are possible
  - **Modulate:** Use these colour values to modify the color of the surface that might have been determined by a lighting model
  - **Replace:** To assign a color to the surface based on only the texture value. This color assignment is carried out as part of the assignment of fragment colours.



Challenges in Texture Mapping

1. **Forward Mapping Functions:** Determine the map from texture coordinates to object coordinates. A two-dimensional texture usually is defined over a rectangular region in texture space. The mapping from this rectangle to an arbitrary region in three-dimensional space may be a complex function or may have undesirable properties. For example, if we wish to map a rectangle to a sphere, we cannot do so without distortion of shapes and distances.
2. **Backward Mapping**: Owing to the nature of the rendering process, which works on a pixel-by-pixel basis, it would be beneficial to get the inverse map from screen coordinates to texture coordinates. When determining the shade of a pixel, it can be determined what point in the texture image to use—a calculation that requires to go from screen coordinates to texture coordinates.
3. **Aliasing**: Because each pixel corresponds to a small rectangle on the display, it is not mapping points to points, but rather areas to areas. Here again is a potential aliasing problem that must be treated carefully to avoid artifacts, such as wavy sinusoidal or moiré patterns.

Linear Texture Mapping

Mapping Functions

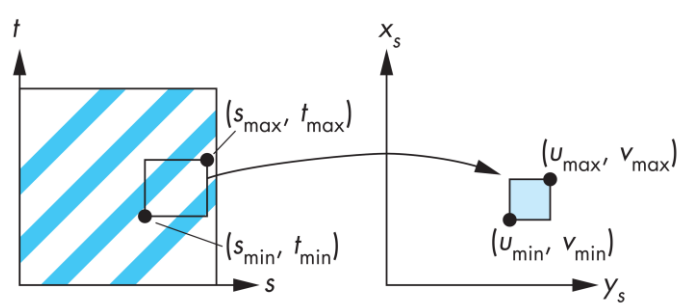
- A point **p** on the surface is a function of two parameters u and v. For each pair of values, we generate the point,

$$\boldsymbol{p}(u,v) = \begin{bmatrix} x(u,v) \\ y(u,v) \\ z(u,v) \end{bmatrix}$$

- The point on the texture map T(s, t) can be mapped to a point on surface $\boldsymbol{p}(u,v)$ by a linear map

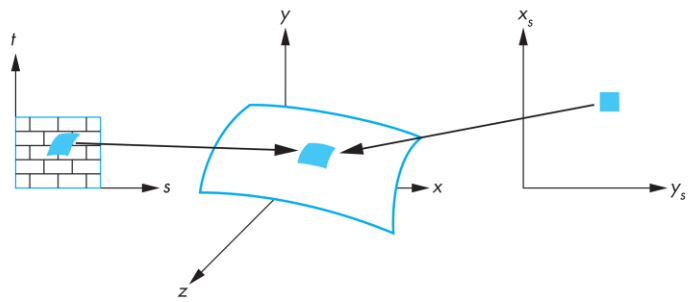
$$u = as + bt + c$$
$$v = ds + et + f$$

- Linear mapping makes it easy to map a texture to a group of parametric surface patches
- But it does not take into account the curvature of the surface. Equal-sized texture patches must be stretched to fit over the surface patch.



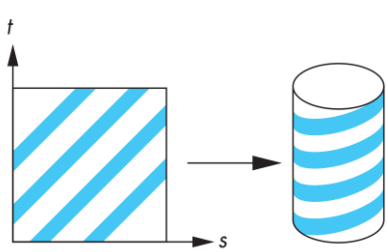
Inverse Mapping

- When computing a color for the square pixel centered at screen coordinates (x_s, y_s), corresponds to a point (x, y, z) in object space
- But, if the object is curved, the projection of the corners of the pixel backward into object space yields a curved **preimage** of the pixel.
- In terms of the texture image $T(s, t)$, projecting the pixel back yields a preimage in texture space that is the area of the texture that ideally should contribute to the shading of the pixel.



Two-part Mapping

1. Maps the texture to a simple 3D intermediate surface, such as a sphere, cylinder, or cube.



Texture mapping with a cylinder

- Suppose that the texture coordinates vary over the unit square, and that a cylinder of height *h* and radius *r* is used as an intermediate object.
- Point on cylinder are given by parametric equations

$$x = rcos(2\pi u)$$
$$y = rsin(2\pi u)$$
$$z = v/h$$

- As u and v vary over $[0,1]$

$$s = u$$
$$t = v$$

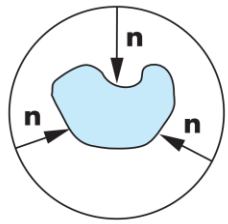
- By using only the curved part of the cylinder, and not the top and bottom, it is possible to map the texture without distorting its shape.

Two-part Mapping

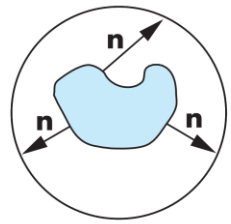
Mapping Functions

2. The intermediate surface containing the mapped texture is mapped to the surface being rendered.

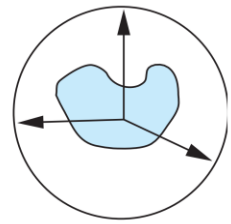
- Map the texture values on the intermediate object to the desired surface.
- Can be done in three ways. Top view with cylindrical intermediate surface and blue shaded object is shown.



Go from the texture value at a point on the intermediate object in the direction of the normal until it intersect the object, and then place the texture value at the point of intersection.



Starting at a point on the surface of the object and going in the direction of the normal at this point until it intersect the intermediate object, From where the texture value is obtained



If the centre of the object is known. Draw a line from the centre through a point on the object, and to calculate the intersection of this line with the intermediate surface

17

Texture Map

Mesh

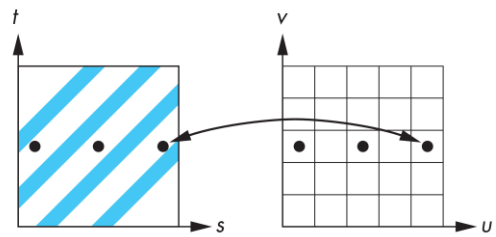
UV Map

Texture Map

18

Texture Mapping: Challenge of Aliasing

- If there is a repeated texture to be mapped to a flat surface.
- The back projection of the centre of each pixel happens to fall in between the dark lines, and the texture value is always the lighter color.



19

How to Generate/Render Realistic Images

20

Modelling Transparency

- How do we deal with transparent objects?
- Need to model Refraction.
- Ignore them! Leads to non-refractive transparency, which is still OK.
- Refraction takes considerable effort to handle. We will see it later.
- Other tricks: Interpolated transparency

Interpolated Transparency

- When two objects overlap at a pixel, use a weighted average of the two colours at the pixel.

$$I_\lambda = (1 - k_{t1})\mathrm{I}_{\lambda 1} + k_{t1}\mathrm{I}_{\lambda 2}$$

- k gives the transmission coefficient or the transparency of each polygon.
- If $k_{t1} = 0$, polygon 1 is totally opaque, $(1 - k)$ is opacity.

Thank You

Anonymous Course Feedback